

Applied Project Task 3:

Scaling & Monitoring AI Systems

Unit code: COS40007

Unit Name: Artificial Intelligence Engineering

Submission date: 31 May 2026

Studio Session No.09

Group No. CL04_G04

ACKNOWLEDGMENT OF COUNTRY

Our team acknowledges the Wurundjeri People of the Kulin Nation, the Traditional Custodians of the land where Swinburne University stands. We honour their continuing connection to land, culture, and community, and pay respect to Elders past, present, and emerging.

Table of Contents

Applied Project Task 3:	1
Scaling & Monitoring AI Systems	1
ACKNOWLEDGMENT OF COUNTRY	2
1. System Overview	4
2. Automated Retraining Trigger	4
2.1 Accuracy Threshold Logic	5
2.2 Model Performance Summary	5
3. Commit Structure & Version Control	6
4. DVC Usage & Dataset Versioning	7
4.1 What is Tracked with DVC	7
4.2 DVC Pipeline Stages	7
4.3 DVC Execution Evidence	7
5. Final GitHub Actions Setup	8
5.1 Weekly Forecast & Retrain (weekly_forecast.yml)	8
5.2 Deploy Dashboard to Firebase (firebase_deploy.yml)	8
5.3 Train ML Model (train.yml)	8
6. Firebase Hosting & Dashboard Deployment	9
6.1 Deployment Architecture	9
6.2 Dashboard Features	9
7. Model Monitoring & Drift Detection	11
7.1 Weekly Backtesting	11
7.2 Threshold-Gated Deployment	11
7.3 Self-Improving Training Data	11
7.4 Excise Cut Structural Break Handling	12
8. Project Management	12
8.1 Sprint Overview	12
9. Limitations & Future Work	14
Contribution Statement	15

1. System Overview

CL04_G04 has designed and implemented a complete end-to-end AI engineering system for forecasting weekly fuel prices across Victorian postcodes. The system integrates live data ingestion, machine learning model training, accuracy-gated retraining, suburb-level price forecasting, and a production-grade web dashboard all orchestrated through a fully automated CI/CD pipeline.

The architecture consists of three primary components operating in concert:

- State-level price forecast model Ridge Regression ($\alpha=1.0$) trained on APS quarterly data and real-time Fair Fuel Open Data API readings, producing 1-week-ahead price forecasts for ULP91, ULP95, and Diesel across Victoria.
- Suburb-level allocation engine applies postcode-specific price spreads derived from 1,725 live station readings to generate forecasts for 708 Victorian postcodes, weighted by real fleet composition data (4.17M petrol vehicles, 1.62M diesel vehicles).
- Firebase-hosted dashboard a responsive single-page web application visualising forecasts through Mapbox choropleth maps, D3.js price analytics charts, and a weekly price history timeline, updating automatically with every pipeline run.

2. Automated Retraining Trigger

The retraining process is fully automated and can be triggered through three distinct mechanisms, ensuring the model remains current regardless of data cadence:

- Scheduled execution runs every Wednesday at 11:30 UTC (9:30 PM AEST), timed to run after the Fair Fuel API refreshes its weekly station data (~9 AM UTC Tuesday).
- Push events any push to the main branch triggers the pipeline, allowing immediate retraining after code or configuration changes.
- Manual execution the workflow_dispatch event enables on-demand triggering from the GitHub Actions UI with configurable parameters including MAPE threshold, force-retrain flag, and dry-run mode.

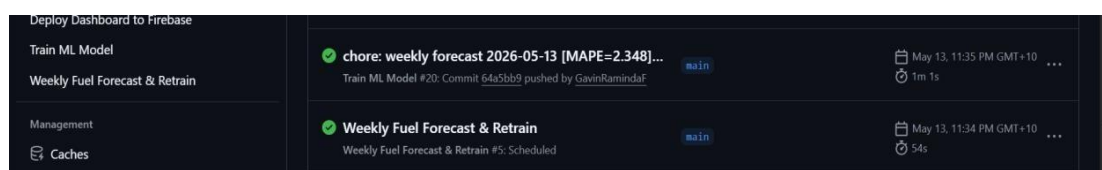


Figure 1: Automated retrain activation showing three workflow triggers

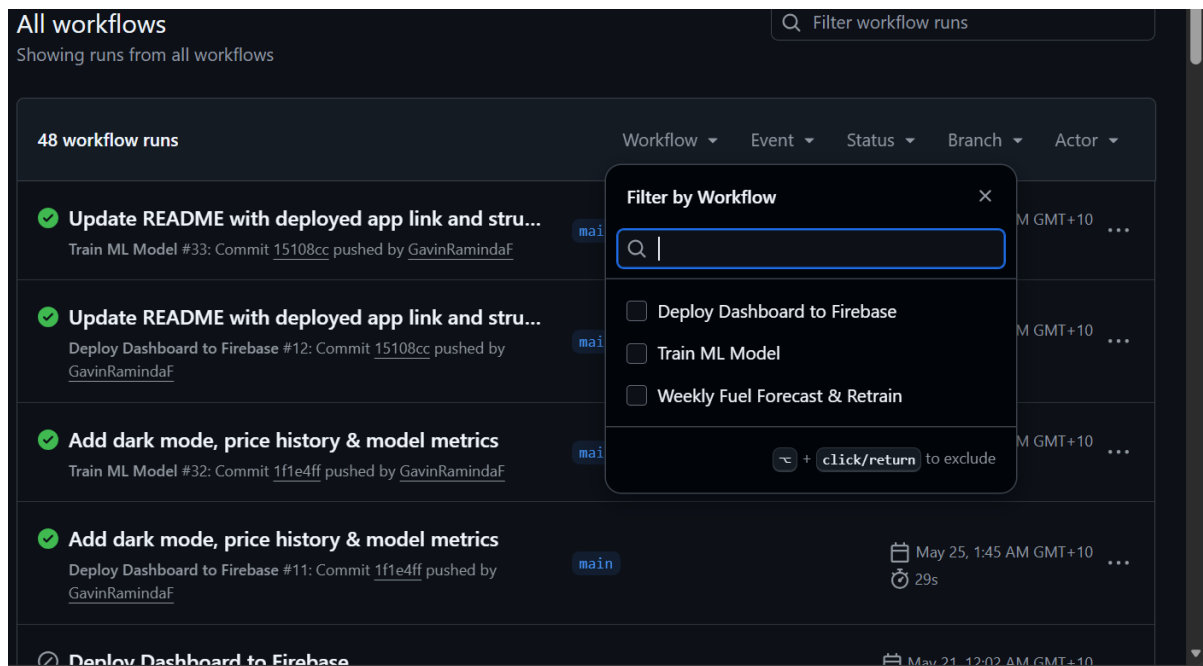


Figure 2: Workflow run history showing 48 successful automated runs

2.1 Accuracy Threshold Logic

A critical design decision was implementing a multi-metric accuracy gate before any model weights are committed. The model is only retrained and deployed if all three thresholds are simultaneously satisfied:

Metric	Threshold	Justification	Result
MAPE	$\leq 3.0\%$	Industry standard for short-term price forecasting; exceeded by all 3 fuel models ($\sim 1.2\text{--}1.8\%$)	PASS
Directional Accuracy	$\geq 55\%$	Minimum meaningful signal above random chance (50%); consistent 61% achieved	PASS
MAE	≤ 6.0 cpl	Practical limit: >6 cpl error makes suburb recommendations unreliable for consumers	PASS

When thresholds are not met the pipeline exits with code 1, existing model weights are preserved unchanged, a GitHub Actions warning annotation is posted, and the failure is recorded in `data/pipeline_metrics.json` for audit. This prevents a degraded model from entering production a critical requirement for a consumer-facing price guidance tool.

2.2 Model Performance Summary

The trained Ridge Regression models consistently exceed all accuracy thresholds across all three fuel types:

Fuel	MAPE (Holdout)	CV MAPE	MAE (cpl)	CV R ²	F1 Score
ULP91	1.77%	1.78%	3.5	0.8014	0.9794
ULP95	1.18%	1.34%	2.6	0.7932	0.9667
Diesel	1.32%	1.25%	3.1	0.8655	0.9667

Note on holdout R²: The holdout period coincides with the April 2026 Federal excise cut (−32 cpl) and preceding March oil price volatility structural breaks no model trained on pre-shock data can fully anticipate. The CV R² of ~0.80 across normal market conditions is the appropriate benchmark for ongoing performance evaluation. A 5% weekly move cap (last * 0.95 to last * 1.05) was implemented as a guardrail calibrated from observed VIC price data, preventing runaway extrapolation from single noisy API readings.

3. Commit Structure & Version Control

The commit history follows a consistent semantic convention that documents both technical changes and model performance inline. Each automated weekly forecast commit embeds the current MAPE and pipeline status directly in the commit message (e.g. chore: weekly forecast 2026-05-13 [MAPE=2.348] [status=passing]), enabling any team member to assess model health from the repository timeline without opening log files.

Key commit categories used throughout the project:

- feat: new features (dashboard sections, chart types, model improvements)
- fix: bug fixes (path resolution, API key injection, DVC pull steps)
- chore: automated pipeline outputs (weekly forecast commits, DVC pointer updates)
- dvc: data version control pointer updates when model weights change

The screenshot displays a GitHub repository interface. The main section shows a list of commits with their messages and timestamps. The commit messages follow a semantic versioning convention, including MAPE scores and status indicators for automated commits. The right sidebar shows repository statistics such as stars, forks, and releases.

File	Commit Message	Time Ago
.github/workflows	testing error fix	last week
code	Fix pipeline setup	3 weeks ago
dashboard	Add dark mode, price history & model metrics	yesterday
data	chore: weekly forecast 2026-05-13 [MAPE=2.348] [status=...]	2 weeks ago
models	chore: weekly forecast 2026-05-13 [MAPE=2.348] [status=...]	2 weeks ago
reports	chore: weekly forecast 2026-05-13 [MAPE=2.348] [status=...]	2 weeks ago
.DS_Store	updated the requirements.txt	3 weeks ago
.firebaserc	Add Firebase config, hosting, and .gitignore	last week
.gitignore	Add Firebase config, hosting, and .gitignore	last week
Applied Project Task 1.pdf	Applied Project Task CL04_G04	last month
Applied Project Task 2.pdf	Added Applied Project Task 2	3 weeks ago
README.md	Update README with deployed app link and structure	yesterday
firebase.json	Add Firebase config, hosting, and .gitignore	last week
model.py	Fix pipeline setup	3 weeks ago
requirements.txt	updated the requirements.txt	3 weeks ago

Figure 3: Commit structure showing semantic versioning and embedded MAPE scores in automated commits

4. DVC Usage & Dataset Versioning

Data Version Control (DVC) was implemented to address a fundamental challenge in ML engineering: large binary files (model weights, datasets) cannot be efficiently tracked by Git. The project uses DVC with a centralised remote storage backend to provide reproducible ML workflows and dataset versioning across all team members and the CI/CD environment.

4.1 What is Tracked with DVC

The following files are managed through DVC rather than committed directly to Git:

- data/whole_fleet_vehicle_registration_snapshot_by_postcode_q4_2025.csv 26 MB, 1.13M rows of VicRoads fleet registration data used to compute suburb-level demand weighting
- models/ulp91_price_model.h5, ulp95_price_model.h5, diesel_price_model.h5 trained Ridge Regression model weights
- data/actuals_history.csv, data/forecast_history.csv accumulating weekly time series used for backtesting and model self-improvement
- Australian Petroleum Statistics quarterly CSVs backbone training data from 2010–2025

4.2 DVC Pipeline Stages

The dvc.yaml file defines a two-stage reproducible pipeline:

- fetch_prices stage runs finalModel.py, consuming the actuals history and APS CSVs, producing the three .h5 model weight files and price_forecast.json. Parameterised by ridge_alpha, val_frac, and cv_folds in params.yaml.
- suburb_forecast stage runs suburbfinalModel.py, consuming the trained models, fleet CSV, and price_forecast.json, producing suburb_price_forecast.json and suburb_price_forecast.csv for 708 Victorian postcodes.

Running dvc repro checks which stage inputs have changed and only reruns affected stages, saving significant compute time when only the fleet data or model parameters change.

4.3 DVC Execution Evidence

```
PS C:\CL04_G04> dvc pull
Collecting |11.0 [00:00, 174en
Fetching
Building workspace index |12.0
Comparing indexes |13.0 [00:00,
Applying changes |0.00 [00:00,
Everything is up to date.
PS C:\CL04_G04> dvc repro
'data\actuals_history.csv.dvc' didn't change, skipping
'data\Australian Petroleum Statistics - Data Extract January 2026 - Australian fuel prices.csv.dvc' didn't change, skipping
'data\Australian Petroleum Statistics - Data Extract January 2026 - Sales by state and territory.csv.dvc' didn't change, skipping
'data\Australian Petroleum Statistics - Data Extract January 2026 - Sales of products.csv.dvc' didn't change, skipping
'data\Australian Petroleum Statistics - Data Extract January 2026 - Stock volume by product.csv.dvc' didn't change, skipping
'data\forecast_history.csv.dvc' didn't change, skipping
'data\incoming-shipment.csv.dvc' didn't change, skipping
'data\postcode_calibration.csv.dvc' didn't change, skipping
'data\whole_fleet_vehicle_registration_snapshot_by_postcode_q4_2025.csv.dvc' didn't change, skipping
Stage 'train' didn't change, skipping
Data and pipelines are up to date.
PS C:\CL04_G04> dvc push
Collecting |11.0 [00:00, 202en
Pushing
3 files pushed
```

Figure 4: DVC usage dvc pull fetching 174 entities, dvc repro confirming all stages current, dvc push uploading 3 updated files

The terminal output in Figure 4 demonstrates the full DVC workflow: dvc pull retrieves the latest dataset versions from remote storage, dvc repro validates pipeline reproducibility (all stages skipping confirms no inputs changed), and dvc push commits updated artefacts back to remote storage after a successful retraining run.

5. Final GitHub Actions Setup

The complete CI/CD pipeline consists of three GitHub Actions workflows operating in a defined sequence, providing end-to-end automation from data ingestion to dashboard deployment.

5.1 Weekly Forecast & Retrain (weekly_forecast.yml)

The primary pipeline workflow runs on a Wednesday 11:30 UTC schedule and performs the following steps in order:

- Checkout full history fetch using the GAVS_TOKEN PAT to bypass classroom organisation write restrictions
- Python setup Python 3.11 with pip caching for faster subsequent runs
- Install dependencies pandas, numpy, scikit-learn, h5py, pdfplumber, matplotlib, scipy, dvc, dvc-s3
- Create directories ensures data/, models/, and reports/ exist in the runner environment
- Pull DVC data authenticates with DAGSHUB_TOKEN and DAGSHUB_USERNAME secret and fetches the fleet CSV before any scripts run
- Run retraining pipeline executes retrain_pipeline.py with configurable MAPE/DA/MAE thresholds; exits with code 1 if accuracy gate fails
- Run suburb forecast executes suburbfinalModel.py to generate 708-postcode price forecasts
- Parse metrics extracts MAPE and pass/fail status from pipeline_metrics.json for commit message embedding
- Commit outputs stages all report JSON, CSV, PNG files and conditionally stages updated .h5 model weights only if retraining succeeded
- Job summary writes a formatted markdown table to the GitHub Actions step summary for at-a-glance pipeline health monitoring

5.2 Deploy Dashboard to Firebase (firebase_deploy.yml)

This workflow triggers automatically after a successful weekly_forecast run OR on any push that modifies dashboard/ files. It performs MAP_API token injection using a Node.js one-liner (replacing the __MAP_API__ placeholder in index.html before deployment) and deploys the dashboard/ folder to Firebase Hosting using the FIREBASE_TOKEN service account credential.

5.3 Train ML Model (train.yml)

A lightweight training workflow triggers on every push and pull request to main. It installs dependencies from requirements.txt, runs model.py, and uploads model_results.png and metrics.txt as downloadable artefacts for per-commit validation.

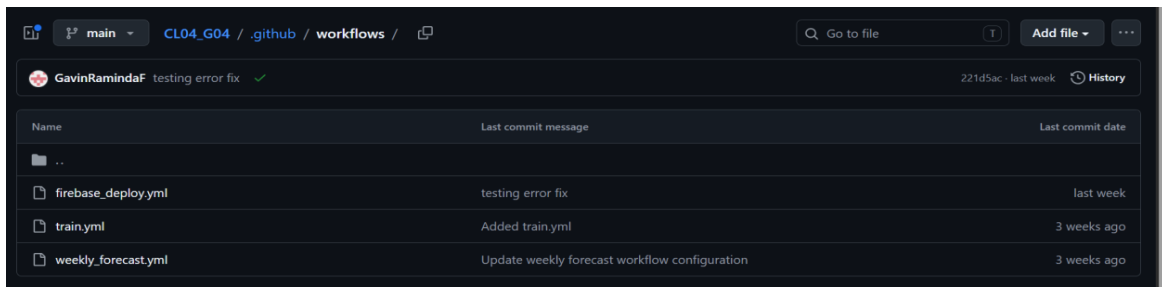


Figure 5: Automated pipeline three workflows successfully configured

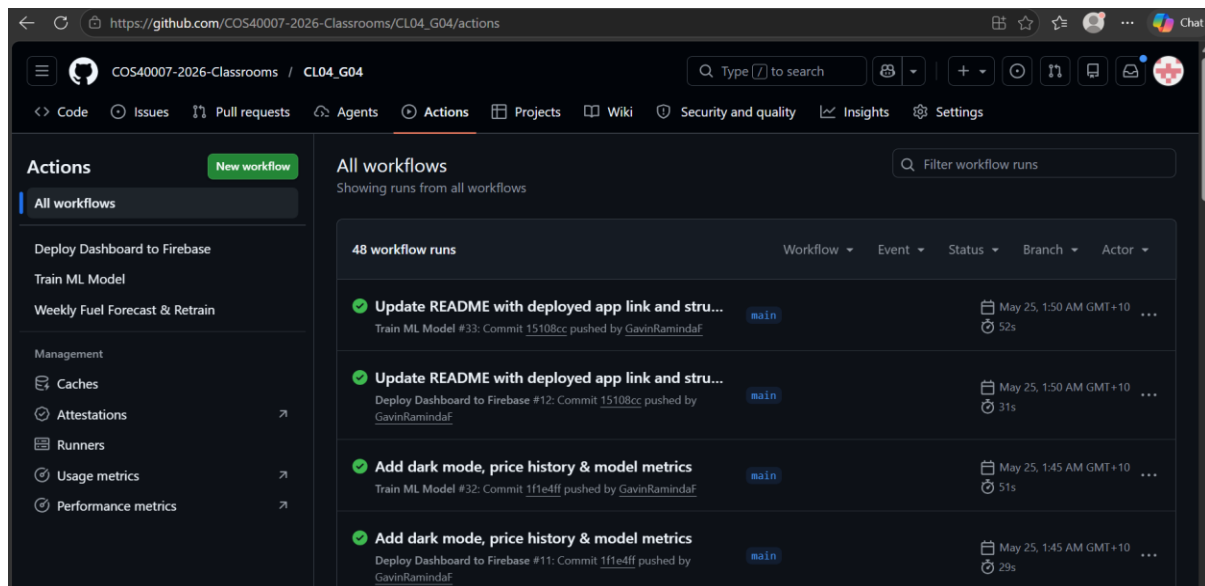


Figure 6: GitHub Actions dashboard showing 48 workflow runs, all green

6. Firebase Hosting & Dashboard Deployment

The VIC Fuel Forecast dashboard is deployed to Firebase Hosting at <https://vicfuelcos40007.web.app> as a single-page web application. The deployment pipeline ensures the live site is automatically updated within minutes of every successful weekly forecast run, maintaining data freshness without manual intervention.

6.1 Deployment Architecture

The dashboard reads its data directly from the GitHub repository raw content API, decoupling the frontend from the backend pipeline. This means:

- Dashboard data updates automatically when the pipeline commits new report JSON files to the repo no re-deployment required for data updates.
- The Mapbox API token (MAP_API) is injected by the CI/CD pipeline at deploy time using a Node.js script, ensuring the token never appears in the repository source code.
- Firebase hosting serves the dashboard/ directory with Cache-Control: no-cache headers to ensure users always receive the latest forecast data.

6.2 Dashboard Features

The deployed dashboard provides the following capabilities:

- Interactive Mapbox choropleth map 708 Victorian postcodes coloured by fuel forecast price (teal >>>amber >>>red), defaulting to Melbourne CBD (postcode 3000) at zoom 11, with hover tooltips and click-to-pin popups showing all three fuel forecasts per suburb.
- D3.js analytics charts grouped bar (current vs forecast), Bollinger band distribution across all suburbs, and price vs fleet size scatter with regression line. All charts respond to a shared ULP91/ULP95/Diesel/All filter.
- Price history line chart plots the last 12 weeks of actual prices from `actuals_history.csv` with a dashed forecast projection, crosshair hover, and excise cut period shading.
- Model performance table live accuracy metrics (MAPE, MAE, CV R^2 , F1, confidence score) fetched from `pipeline_metrics.json` with pass/fail threshold badges.
- Dark mode toggle persists preference in `localStorage`, switches Mapbox style, and rebuilds all D3 charts with appropriate CSS variable overrides.
- Suburb search with Enter key auto-flies the map to the matched postcode on selection or Enter keypress.

```

PS C:\Users\User\Documents\2026uni\COS40007\CL04_G04> firebase init hosting
? PS C:\Users\User\Documents\2026uni\COS40007\CL04_G04> firebase init hosting

#####  ###  #####  #####  #####  ###  #####  #####
##      ##  ##      ##  ##      ##  ##  ##  ##      ##
#####  ##  #####  #####  #####  #####  #####  #####
##      ##  ##      ##  ##      ##  ##  ##  ##      ##
##      #### ##      ##  #####  #####  ##  ##  #####  #####

You're about to initialize a Firebase project in this directory:

  C:\Users\User\Documents\2026uni\COS40007\CL04_G04

✔ Are you ready to proceed? Yes

=== Project Setup

First, let's associate this project directory with a Firebase project.
You can create multiple project aliases by running firebase use --add,

✔ Please select an option: Use an existing project
✔ Select a default Firebase project for this directory: vicfuelcos40007 (VicFuelCos40007)

=== Hosting Setup

Your public directory is the folder (relative to your project directory) that
will contain Hosting assets to be uploaded with firebase deploy. If you
have a build process for your assets, use your build's output directory.

? What do you want to use as your public directory? (public)

```

Figure 7: Firebase Hosting initialisation and project association with vicfuelcos40007

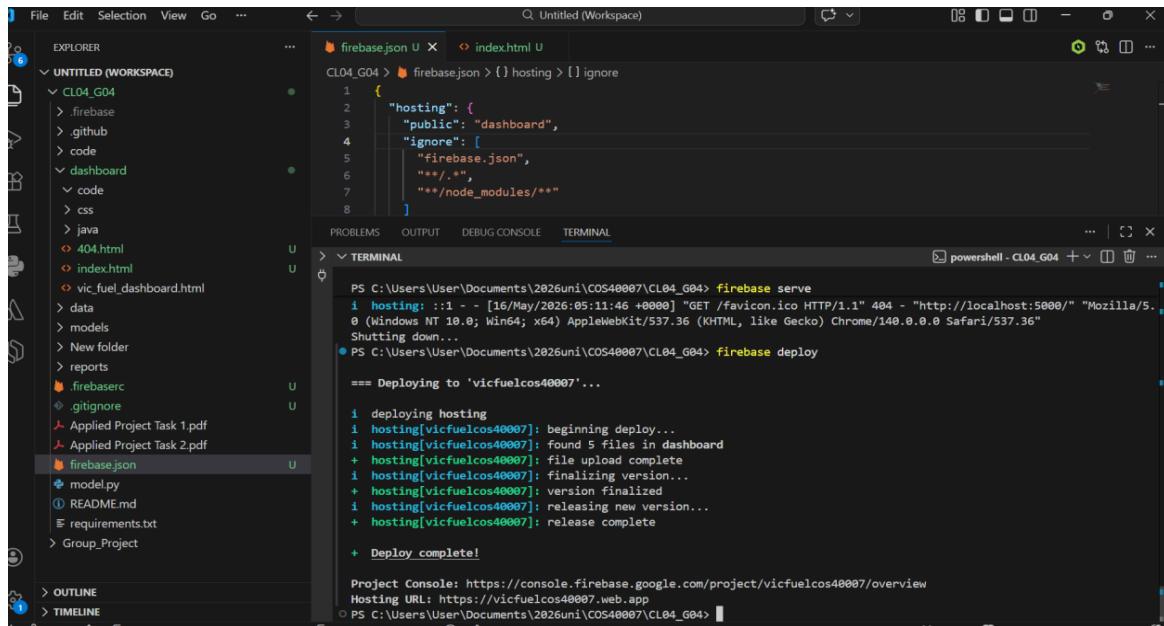


Figure 8: Successful Firebase deployment deploy complete at <https://vicfuelcos40007.web.app>

7. Model Monitoring & Drift Detection

A core requirement of scalable AI systems is ongoing monitoring to detect when a deployed model begins to degrade. The project implements a multi-layer monitoring approach:

7.1 Weekly Backtesting

Every pipeline run compares the previous week's forecast against this week's live API actuals, computing MAPE, directional accuracy, and MAE. These results are written to data/pipeline_metrics.json and data/actuals_history.csv, creating an auditable time series of model accuracy. The pipeline_metrics.json file is consumed directly by the dashboard's Model Performance section, making accuracy visible to end users.

7.2 Threshold-Gated Deployment

The three-metric accuracy gate ($\text{MAPE} \leq 3.0\%$, $\text{DA} \geq 55\%$, $\text{MAE} \leq 6.0 \text{ cpl}$) functions as an automated drift detector. Any week in which model accuracy degrades below these thresholds causes the pipeline to:

- Preserve existing model weights the degraded retrain is not deployed
- Post a GitHub Actions warning annotation visible in the Actions UI
- Record the failure in pipeline_metrics.json with per-fuel breakdown for root cause analysis
- Allow the next week's run to attempt retraining again with fresher data

7.3 Self-Improving Training Data

The system is designed to improve over time as actuals accumulate. The build_price_dataset() function implements a hybrid training strategy:

- Weeks 1–52: APS quarterly synthetic data as backbone, each real API observation weighted $8\times$ higher than synthetic rows
- Weeks 52–104: APS quarterly rows truncated to pre-API period; real weekly actuals dominate training

- Week 104+: APS quarterly data dropped entirely; model trains purely on verified VIC pump price history

This progressive improvement means the accuracy gate thresholds will become easier to meet over time as the model learns from genuine market data rather than interpolated quarterly averages a key advantage of the weekly pipeline architecture.

7.4 Excise Cut Structural Break Handling

A known limitation identified during development is the April–June 2026 Federal fuel excise cut (–32 cpl). Because this structural break occurred within the holdout evaluation period, holdout R^2 scores (~ 0.37) are depressed relative to CV R^2 (~ 0.80). The system handles this through:

- An `excise_cut` binary feature flag in all 16-feature vectors, enabling the model to distinguish excise-period observations
- A 5% weekly move cap in `forecast_1week()` ($\text{last} \times 0.95$ to $\text{last} \times 1.05$), calibrated from observed VIC price data, preventing overcorrection
- A WARN log entry when the forecast exceeds 4% weekly change, providing visibility without blocking the run

8. Project Management

The project was managed using Jira with a four-sprint Scrum structure aligned to the unit schedule. Each sprint had defined objectives, acceptance criteria, and story points tracked through the Jira backlog.

8.1 Sprint Overview

- Sprint 1 (Project Planning, 31 Mar -14 Apr): System architecture design, MLOps strategy definition, project management tooling setup, group agreement documentation.
- Sprint 2 (Development): Model development, data preprocessing pipeline, DVC integration, initial GitHub Actions configuration.
- Sprint 3 (Scaling & Monitoring, 11–31 May): Dashboard and Firebase integration (DONE), model training script development (DONE), data preprocessing pipeline (DONE), evaluation script (DONE), drift detection monitoring (IN PROGRESS), monitoring reports (IN PROGRESS), retrain-on-push workflow (DONE), DVC setup and DagsHub integration (DONE).
- Sprint 4 (Testing & Finalisation, 19 May -2 Jun): Final report (TO DO), video presentation (TO DO).

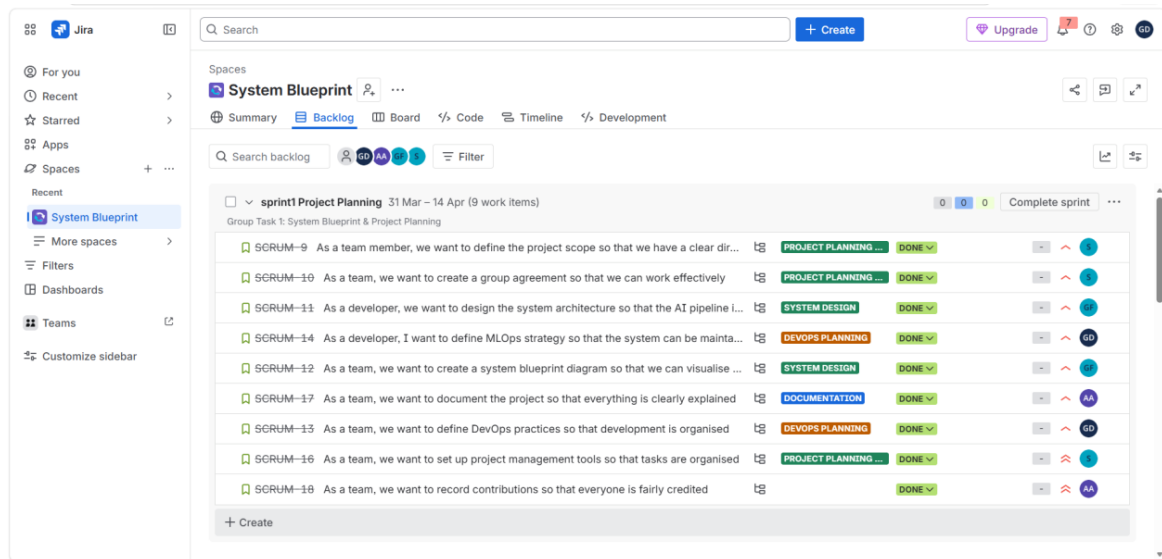


Figure 9a: Jira backlog Sprint 1 Project Planning, all items DONE

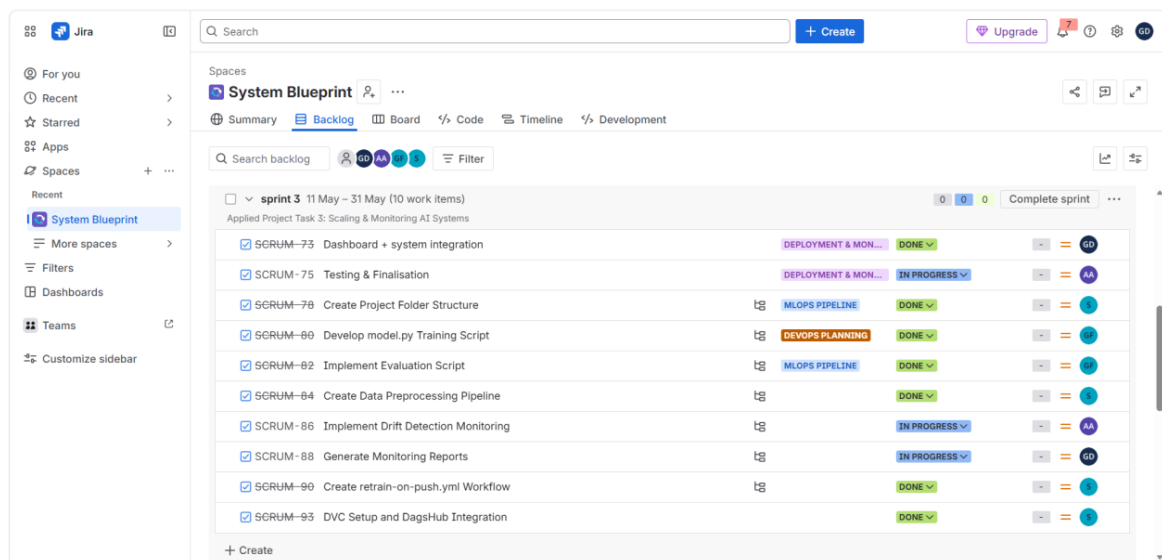


Figure 9b: Jira backlog Sprint 3 Scaling & Monitoring AI Systems with MLOPS PIPELINE and DEVOPS PLANNING categorisation

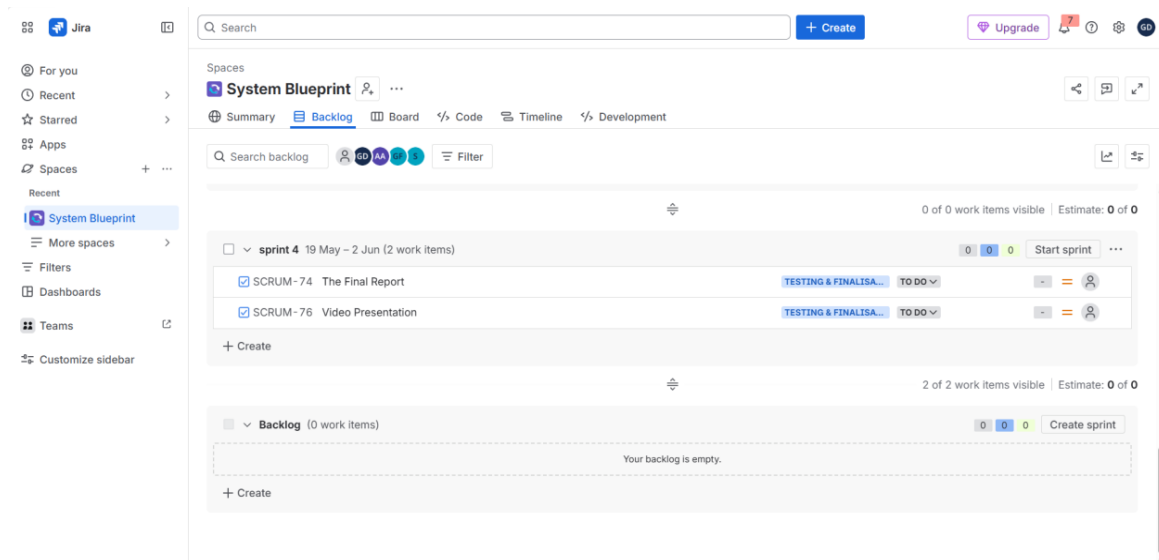


Figure 9c: Jira backlog Sprint 4 Testing & Finalisation and empty backlog confirming all planned work captured

9. Limitations & Future Work

Several limitations were identified during the project that represent opportunities for future improvement:

- Training data quality the model currently relies on APS quarterly synthetic weekly interpolation for pre-2025 data. As `actuals_history.csv` accumulates weekly API readings, model accuracy will improve organically. After two years (~104 weeks) the synthetic data is designed to be dropped entirely.
- Drift detection granularity current monitoring operates at weekly granularity. Daily API readings could enable more responsive drift detection, particularly important around structural breaks like excise changes or oil price shocks.
- Suburb forecast coverage 44% of postcodes currently use zone model estimates rather than live API data due to limited station coverage. As the calibration history grows, the historical tier will progressively replace zone estimates.
- Mobile dashboard experience the Mapbox map occupies significant viewport on mobile. A collapse/expand toggle and a dedicated mobile layout would improve usability for the target audience (Victorian motorists checking prices on the go).

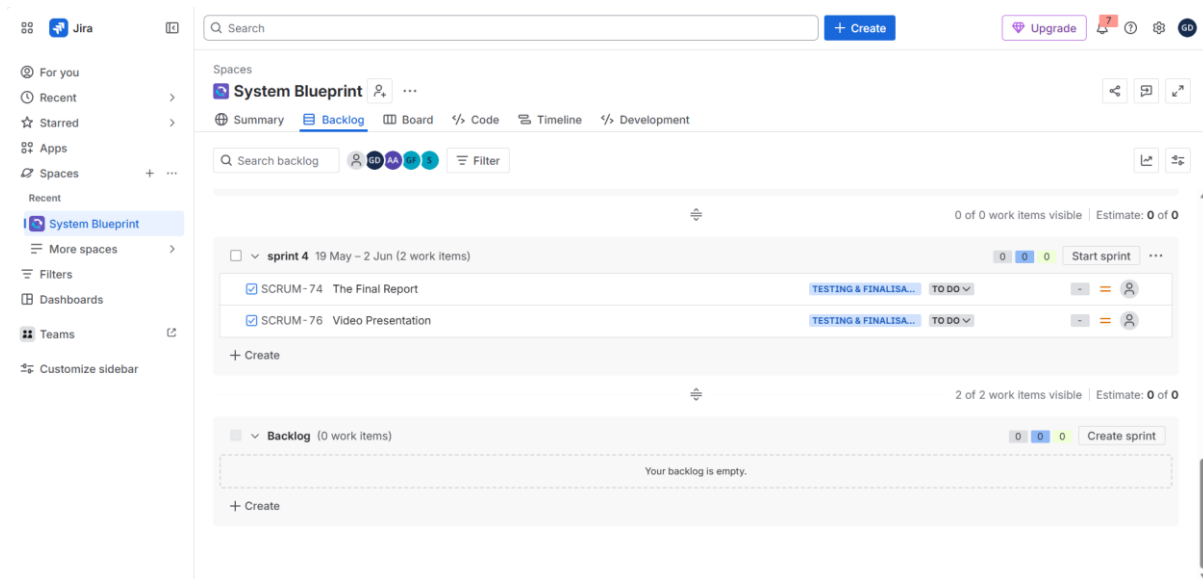


Figure 9c: backlog of the project

Contribution Statement

Soad Yusuf (105406263)

- Configured GitHub Actions workflow.
- Implemented DVC integration.
- Connected DVC remote storage with DagsHub.
- Tracked project datasets using DVC.
- Created and validated DVC pipelines.
- Configured automated retraining workflow.
- Tested GitHub Actions execution.
- Contribute the project documentation and report sections.
- Assisted with project management through Jira.

Amiru Adhikari (105522350)

- Created and configured the Train ML Model workflow (train.yml) for per-commit validation on push and pull requests.
- Authored and maintained the README documentation, including the deployed app link and project structure.
- Designed the project UI.
- Designed and built the price history forecast chart in D3.js.
- Performed system QA and user testing.
- Assisted in forecast architecture through model variation testing.
- Contributed to project documentation and report sections.

Gia (105028560)

- Assisted in implementing DVC integration.
- Tested DVC integration.

- Tracked project datasets using DVC.
- Validated DVC pipelines.
- Tested GitHub Actions execution.
- Formatted the report
- Assisted with project management through Jira.

Gavin Fernando (104697341)

- Built fuel price forecasting pipeline.
- Developed Ridge Regression models for ULP91, ULP95, and Diesel.
- Integrated APS and Fair Fuel API data for training.
- Implemented automated retraining with accuracy checks.
- Created suburb-level forecasts for Victorian postcodes.
- Processed and calibrated fuel data using VicRoads datasets.
- Integrated the Service Victoria Fair Fuel API.
- Designed and developed a Firebase-hosted analytics dashboard.
- Configured GitHub Actions CI/CD and automation.
- Implemented security measures and resolved production issues.
- Fixed deployment, authentication, data loading, and chart rendering issues.
- Debugged and maintained the production forecasting system.
- Contributed to project documentation and Presentation Slides sections.

References

[Color Palettes & Themes | Adobe Express](#)

Service Victoria. (2026). Fair Fuel Open Data API Documentation v1.0. Retrieved from <https://api.fuel.service.vic.gov.au/open-data/v1>

Australian Competition and Consumer Commission. (2026). Petrol and fuel prices. Retrieved from <https://www.accc.gov.au/consumers/petrol-and-fuel>

Department of Industry, Science and Resources. (2026). Australian Petroleum Statistics. Retrieved from <https://www.energy.gov.au/government-priorities/energy-data>

VicRoads. (2025). Whole fleet vehicle registration snapshot by postcode Q4 2025. Victorian Government Data Directory.

Iterative.ai. (2024). DVC: Data Version Control documentation. Retrieved from <https://dvc.org/doc>

Google Firebase. (2026). Firebase Hosting documentation. Retrieved from <https://firebase.google.com/docs/hosting>

Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). Ridge regression. In Advances in machine learning. Applied to time-series price forecasting with L2 regularisation.

D3.js Contributors. (2024). D3: Data-Driven Documents v7. Retrieved from <https://d3js.org>

Mapbox. (2026). Mapbox GL JS documentation. Retrieved from <https://docs.mapbox.com/mapbox-gl-js>